PROBABILISTIC VIEW OF OPTIMIZATION ALGORITHMS FOR TRAINING MACHINE LEARNING MODELS

ABDULAZIZ SOBIROV
MATH 154: PROBABILITY THEORY

1. Introduction

You have most probably heard some combination of the words 'Machine Learning,' 'Artificial Intelligence' and 'Model' in a single sentence, given the recent explosion in the popularity of open-source Generative AI tools like ChatGPT, Claude and Google Gemini. If you are someone interested in *how* these models work under the general umbrella of Machine Learning theory, you have probably also heard the phrase 'Training a Model' before. In this paper I first provide a quick overview of what 'Training a Model' even means, going over concepts such as Loss Minimization and Gradient Descent. Then, I provide justifications rooted in Probability Theory for conventions used in training models on Training Data and why we expect trained models to work on Unseen Data. Specifically, we will examine how the Law of Large Numbers justifies the use of Loss Minimization when training a model and why the Central Limit Theorem justifies using Gradient Descent to minimize loss. Most sources online tell us that that the Central Limit Theorem and Law of Large Numbers justify these choices, however no formal proofs are shown in general. This paper aims to bridge this gap and formalize the connection between the geometric and probabilistic interpretation of Gradient Descent and Loss Minimization.

2. What is Machine Learning

2.1. Overview

Machine Learning is a broad field. According to IBM, it is the subset of Artificial Intelligence focused on algorithms than can "learn" the patterns of training data and, subsequently, make accurate *inferences* about new data. The main motivation behind Machine Learning is that if we train and optimize a model on a training dataset that reflects, to an extent, real world data that we want to make an inference on, then we will be able to make predictions about unseen data. First, before looking at how concepts learned in Math 154 apply to training models, we should first understand what a 'model' and 'training' mean.

2.2. Models, Training and Loss Functions

Definition 2.2.1. A **model** in Machine Learning is a program that can find patterns or make decisions from a previously unseen dataset. Mathematically, a trained model can be represented as $f_{\mathcal{D},\theta} : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X}$ is the set of inputs, $\mathcal{Y}$ is the set of target outputs (e.g. text, an image, etc.), and $\mathcal{D}$ is the training data, a random variable sampled from some unknown distribution $\mathcal{D} \sim F$, and θ is the model's parameters.

Before proceeding with the next definitions, let me formalize notation choices. Throughout this paper, we will use:

$$x \in \mathbb{R}^k\text{: input into the model}$$
$$y \in \mathbb{R}^j\text{: true output (ground truth)}$$
$$\mathcal{D}\text{: Training Dataset}$$
$$\theta \in \mathbb{R}^d\text{: the model's parameters}$$
$$\hat{y}\text{: the model's predicted output, i.e. } \hat{y} = f_\theta(x)\ .$$

Definition 2.2.2. Model Parameters are the learned values (variables) within a machine learning model that determine how it maps input data to outputs. E.g.: the slope m and y-intercept b of a linear map $y = mx + b$ are the parameters of this linear map, since they dictate how the input x is mapped to the output y. Parameters are generally represented $\theta \in \mathbb{R}^d$.

Definition 2.2.3. A **Loss Function** is a function used to measure model performance by calculating the deviation of a model's prediction from the ground truth output. The most common loss functions is *Ordinary Least Squares (OLS)*:

$$\mathcal{L}_{OLS}(\theta) = \sum_{i=1}^{N}(y_i - f_\theta(x_i))^2 = \sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

We write $\ell(x_i, y_i; \theta)$ as a shorthand for the loss incurred by a single pair of training data (x_i, y_i) and model parameters θ

Therefore, we can formalize what optimizing, or training, a model means:

Definition 2.2.4. Training a model refers to the process of adjusting parameters to minimize a chosen Loss Function.

2.3. True vs Empirical Loss

Corollary 2.3.1. *The standard assumption is that the dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ is sampled from an unknown distribution F. Each pair of training data (x_i, y_i) is drawn independently and identically distributed from F:*

$$(x_i, y_i) \overset{i.i.d.}{\sim} F$$

Definition 2.3.2. The **True Loss** of a model with parameters θ is the expected loss over the distribution from which our data $\mathcal{D}$ is sampled, $\mathcal{D} \sim F$:

$$L(\theta) = \mathbb{E}_{(x,y)\sim F}[\ell(x, y; \theta)]$$

This is specifically the measure of how the model f_θ performs in the real world, but since we do not know the distribution F of the data, we cannot compute $L(\theta)$.

Definition 2.3.3. Empirical Loss (Risk) is the average loss over the observed dataset $\mathcal{D}$:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\ell(x_i, y_i; \theta)$$

(Notice the difference in notation: True Loss is $L(\theta)$ while Empirical Loss is $\mathcal{L}(\theta)$).

Therefore, the training objective is finding:

$$\min_{\theta} \mathcal{L}(\theta)$$

This is called **Empirical Risk Minimization**.

2.4. Gradient Descent Algorithm

$\mathcal{L}(\theta)$ defines a surface (a "loss landscape") over the parameter space. Geometrically, optimization is viewed as moving "downhill" in this surface. Since in the real world this surface is high-dimensional and might include saddle-points, complex curvature, and different global and local minima, we cannot calculate a closed-form solution for the θ that minimizes $\mathcal{L}(\theta)$.

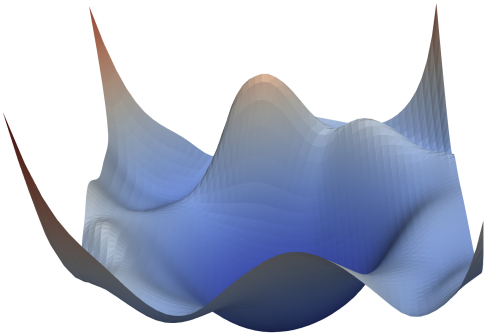


FIGURE 1. Example Loss Landscape of Neural Networks

By Calculus principles, the gradient $\nabla_\theta \mathcal{L}(\theta)$ points in the direction of steepest increase of the loss. So, to decrease the loss we move in the opposite direction: $-\nabla_\theta \mathcal{L}(\theta)$.

Definition 2.4.1. Vanilla Gradient Descent. Since we cannot compute a closed form solution, we update θ iteratively by moving in the direction of the steepest loss:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta)$$

η is the learning rate, a variable that we choose ourselves (the model does not learn it). It controls how "aggressively" we move in the direction $-\nabla_\theta \mathcal{L}(\theta)$.

.

3. The Law of Large Numbers: Why Minimizing Loss Makes Sense

3.1. Convergence and Strong Law

There are two Laws of Large Numbers in Probability theory: Strong and Weak. For the purposes of this paper, we will analyze the Strong Law of Large Numbers. For readers unfamiliar with undergraduate Probability Theory, for understanding this paper it is sufficient to know that Laws of Large Numbers make statements about the stochastic convergence of the sum of a sequence of Random Variables. Probability Theory formalized different notions of what "converge" means, of increasing "strength." As you might have guessed, the Strong Law of Large Numbers guarantees "stronger" convergence (called *almost everywhere* convergence) than its weaker counterpart, which

guarantees "weaker" convergence (called *convergence in probability*). This basic intuition is enough to understand the Law:

Theorem 3.1.1. ***(Strong Law of Large Numbers)****: Let $(X_n)_{n\geq 1}$ be a sequence of independent and identically distributed (i.i.d.) random variables in $\mathcal{L}^4$ (i.e. $\mathbb{E}[X_n^4] < \infty$). If all all X_n in the sequence have common, finite mean $\mathbb{E}[X_n] = \mu$, then:*

$$\frac{1}{n}\sum_{i=1}^{n} X_i := \frac{S_n}{n} \to \mu \ (almost\ everywhere)$$

Intuitively, the SLLN says that if a sequence of independent and identically distributed random variables with finite mean have the same mean, their sample mean will converge to μ.

3.2. Connection to Empirical Risk Minimization

In Corollary 2.3.1 we formalized that the dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ is sampled from an unknown distribution F and all pairs of training data (x_i, y_i) are independent and identically distributed.

Setup: Recall the per-sample loss function ℓ. By the result above, we can treat per sample losses $\ell(x_i, y_i)$ as *i.i.d.* Random Variables under the unknown distribution F. Now, recall that Empirical Loss is:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell(x_i, y_i; \theta)$$

which is exactly the sample mean of a sequence of independent and i.i.d random variables.

We proceed with a claim on the mean of each random variable:

Proposition 3.2.1. *For fixed θ, we have $\mathbb{E}[\ell(x_i, y_i; \theta)] = m$ for all $i \in \{1, .., N\}$.*

Proof. Firstly, let us formalize that since the pairs (x_i, y_i) are *i.i.d.* under F and $\ell(\cdot, \cdot, \theta)$ is a fixed measurable function for fixed θ, it follows that each $\ell(x_i, y_i; \theta)$ are *i.i.d.* random variables. Since each (x_i, y_i) is identically distributed under F, we have:

$$\mathbb{E}_F[\ell(x_i, y_i; \theta)] = \int_{\mathcal{X}\times\mathcal{Y}} \ell(x, y; \theta) dF(x, y)$$

This value is the same for all pairs in $\mathcal{D}$. Denote this common mean as m. Since any useful machine learning model will have finite loss (otherwise we do not even have a model), it is trivial that the means are finite. □

Applying the Strong Law of Large Numbers. Now, we have all the ingredients to apply the Strong Law of Large Numbers:

We have the sequence $\{\ell(x_i, y_i; \theta)\}_{i=1}^N$ of i.i.d random variables with common mean $\mathbb{E}_F[\ell(x_i, y_i; \theta)] = m < \infty$. Thus, by the Strong Law of Large Numbers, we have that:

$$\mathcal{L}(\theta) := \frac{1}{N}\sum_{i=1}^{N} \ell(x_i, y_i; \theta) \xrightarrow{\text{a.e.}} m$$

Now, notice that

$$m = \int_{\mathcal{X}\times\mathcal{Y}} \ell(x, y; \theta) dF(x, y) = \mathbb{E}_{(x,y)\sim F}[\ell(x, y; \theta)]$$
$$= L(\theta) \text{ (True Loss [Check Def 2.3.2])}.$$

Therefore, the Strong Law of Large Numbers says that the Empirical Loss converges almost everywhere to the True Loss. Thus, minimizing the Empirical Loss on a finite set of data minimizes the True Loss on the actual distribution of the data. This is why Empirical Risk Minimization as an objective makes sense.

Remark: It is important to note that in the real-world we might not achieve independent data. This does not mean our result is useless though: it tells us that Empirical Risk Minimization *is* the right direction for us.

4. The Central Limit Theorem: Why Gradient Descent Works

4.1. Convergence in Distribution and CLT

Before we formalize the Central Limit Theorem, it is necessary to mention that the CLT deals with another notion of convergence of Random Variables, which is *convergence in distribution*. This is the weakest out of the notions of convergence.

Theorem 4.1.1. ***(Central Limit Theorem)****: Given $X_n \in \mathcal{L}^2$ which are i.i.d with mean 0 and finite variance $\sigma^2 > 0$, we have:*

$$\frac{S_n}{\sigma\sqrt{n}} \to \mathcal{N}(0, 1) \text{ (}in\ distribution\text{)}$$

This is a fascinating result. No matter what the distribution of individual random variables X_i is, the normalized sample mean converges in distribution to the the standard Normal Distribution (given the necessary conditions).

4.2. CLT and Mini-Batch Gradient Descent

To refresh your understanding of Gradient Descent as a stochastic process, review section 2.4 and definition 2.4.1. Most modern machine learning systems use Mini-Batch Gradient Descent instead of Vanilla Gradient Descent to optimize compute-cost.

Definition 4.2.1. Mini-Batch Gradient Descent. Instead of calculating the gradient over the entire dataset $\mathcal{D}$, we sample a random subset $B \subset \mathcal{D}$ of size $B < N$ and compute

$$g_B(\theta) = \frac{1}{|B|} \sum_{i\in B} \ell(x_i, y_i; \theta)$$

Then, our update becomes:

$$\theta_{t+1} = \theta - \eta \nabla_\theta g_B(\theta)$$

We saw in (sub)section 2.4 that Vanilla Gradient Descent updates parameters in the direction of minimizing Empirical Loss $\mathcal{L}(\theta)$, and section 3 showed that this process will minimize True

Loss. However, for compute-cost optimization, as we mentioned above, most modern models use Mini-Batch Gradient Descent, since the cost per iteration of Vanilla GD is $O(Nd)$, compared to $O(|B|d)$, where d is the dimension of the parameter vector θ (which means the model has d different parameters).

However, now our question is: does Mini-Batch Gradient Descent also move in the direction of minimizing True Loss?

Notice that the gradient of the True Loss is:

$$\nabla_\theta L(\theta) = \nabla_\theta \, \mathbb{E}_{(x,y)\sim F}[\ell(x, y; \theta)] = \int_{\mathcal{X}\times\mathcal{Y}} \nabla_\theta \ell(x, y; \theta) \, dF(x, y) = \mathbb{E}_F[\nabla_\theta \ell(x, y; \theta)]$$

This is uncomputable since F is unknown. However, observe that $g_B(\theta)$ is precisely a sample mean of per-sample gradients $\nabla_\theta \ell(x_i, y_i; \theta)$ over the mini-batch.

Proposition 4.2.2. *For fixed θ, the per-sample gradients $\nabla_\theta \ell(x_i, y_i; \theta)$ are i.i.d. random variables with common mean $\nabla_\theta L(\theta)$.*

Proof. Since (x_i, y_i) are i.i.d. under F and $\nabla_\theta \ell(\cdot, \cdot\,; \theta)$ is a fixed measurable function for fixed θ, it follows that $\nabla_\theta \ell(x_i, y_i; \theta)$ are i.i.d. Since each (x_i, y_i) is identically distributed under F, we have for all i:

$$\mathbb{E}_F[\nabla_\theta \ell(x_i, y_i; \theta)] = \int_{\mathcal{X}\times\mathcal{Y}} \nabla_\theta \ell(x, y; \theta) \, dF(x, y) = \nabla_\theta L(\theta)$$

which is the same for all $i \in B$. □

Applying the Central Limit Theorem. To apply Theorem 4.1.1, we center the per-sample gradients. Working component-wise, fix any component j and define:

$$Z_i := \nabla_{\theta_j} \ell(x_i, y_i; \theta) - \nabla_{\theta_j} L(\theta)$$

By Proposition 4.2.2, Z_i are i.i.d. with mean 0 and finite variance $\sigma_j^2 > 0$. Letting $S_B = \sum_{i\in B} Z_i$, Theorem 4.1.1 gives:

$$\frac{S_B}{\sigma_j \sqrt{B}} \to \mathcal{N}(0, 1) \quad \text{(in distribution)}$$

Substituting back, this says:

$$\frac{\sqrt{B}}{\sigma_j}\Big(g_{\mathcal{B},j}(\theta) - \nabla_{\theta_j} L(\theta)\Big) \to \mathcal{N}(0, 1) \quad \text{(in distribution)}$$

or equivalently, for large B, the j-th component of the mini-batch gradient is approximately distributed as:

$$g_{\mathcal{B},j}(\theta) \;\approx\; \mathcal{N}\left(\nabla_{\theta_j} L(\theta),\; \frac{\sigma_j^2}{B}\right)$$

Intuitively, this result says that the mini-batch gradient $g_{\mathcal{B}}(\theta)$ is an *unbiased* estimator of the true gradient $\nabla_\theta L(\theta)$: on average, each step of gradient descent points in the correct direction. Also, we can see that the variance of this estimate is σ_j^2/B, which decreases as the mini-batch size B grows. Therefore, the Central Limit Theorem justifies using Mini-Batch Gradient Descent to stochastically move in the direction that minimizes true loss $L(\theta)$

5. Conclusion

This paper served as basic introduction to Machine Learning concepts, formalizing the notions of models, training and Loss Minimization. These results are mostly presented algebraically and interpreted geometrically. This paper introduced readers to two famous theorems in probability theory, the Central Limit Theorem and the Law of Large Numbers, and used these results to give a mathematical and intuitive connection to optimization algorithms used in training models.

References

[1] Tom Goldstein. Visualizing the loss landscape of neural nets. `https://www.cs.umd.edu/~tomg/projects/landscapes/`. University of Maryland. Accessed: 2026.

[2] IBM. What is a loss function? `https://www.ibm.com/think/topics/loss-function`. Accessed: 2026.

[3] IBM. What is machine learning? `https://www.ibm.com/think/topics/machine-learning`. Accessed: 2026.

[4] Oliver Knill. Lecture 15: The strong law of large numbers. `https://people.math.harvard.edu/~knill/teaching/math154_2026/handouts/lecture15.pdf`. Harvard University, Math 154, 2026.

[5] Oliver Knill. Lecture 19: The central limit theorem. `https://people.math.harvard.edu/~knill/teaching/math154_2026/handouts/lecture19.pdf`. Harvard University, Math 154, 2026.